

CopyCheck

Technical white paper | Implementation and developer documentation

Version 1.0 - 6 October 2026

CopyCheck reviews two public Solana wallets and explains how their observed execution results differ. It retrieves swap records through CoinGecko, identifies eligible closed positions in the same token, and decomposes the return gap into entry-price and exit-price effects.

Abstract

Copy trading does not guarantee identical outcomes. Two wallets can trade the same token but accumulate different average entry and exit prices. CopyCheck makes these differences inspectable: users enter two addresses and a UTC date range, read the comparison, and follow transaction evidence. The implementation uses deterministic arithmetic rather than an AI model to generate its findings.

The current product is a retrospective review tool. Its strongest claim is that a difference in observed average prices accounts mathematically for a difference in gross percentage returns. It cannot establish copy intent, diagnose the cause of an execution delay, or reconstruct a complete wallet ledger from swaps alone. Conservative eligibility rules withhold results when the supplied history cannot support them.

This paper describes the inspected source snapshot, its public website workflow, integration contracts, algorithms, operational limits and tests. It includes a saved live example and annotated excerpts. Proposed improvements are explicitly separated from implemented behavior.

Evidence basis	Scope
Source snapshot	ca57fc0758fea5e20fc5f31512b45592fad02a66
Website	https://copycheck-trade-review.mszajnowiec.chatgpt.site/
Validation	14 source tests passed during documentation preparation.
Live example	Saved report created 2026-10-06T09:59:13.891Z; one comparable token.
Document boundary	Source review and saved evidence; not an independent security audit or chain-wide accounting verification.

The companion Developer-reference.html contains this paper plus line-numbered source for the principal modules. The evidence folder preserves the report and screenshots used here. No API key is included.

1. Product purpose and practical use

A review after a trade, with evidence for the difference

A trader following another wallet wants to know whether their observed result matched the wallet they followed. CopyCheck answers a narrower, measurable question: for eligible positions in the returned swaps, how different were the two gross returns, and was the larger arithmetic contribution at entry or exit?

User question	Current answer
Did our outcomes differ?	Compare percentage returns on eligible closed token positions.
Where did the difference appear?	Separate the entry-price effect from the exit-price effect.
Did I enter later?	Show the first-buy timestamp difference; this is an observation, not a diagnosis.
What supports the result?	Inspect normalized fills, USD values, timestamps and explorer links.
How many dollars did I lose?	Not a current output. The headline is a return difference in percentage points, before fees.
Did my bot malfunction?	Not established. Bot logs, quotes and transaction submission evidence are not collected.

How someone could use it

After both positions have closed, the user compares their address with the address they intended to follow. If the result repeatedly shows a worse exit price, they have a reason to inspect exit settings or execution records. If history is incomplete, they can shorten the range and inspect the evidence rather than relying on an unsupported percentage. This is a starting point for investigation; it does not select profitable wallets or prescribe a trade.

What is implemented

The app supports Solana, two addresses per request, an inclusive range of up to 30 UTC calendar days, one observed position per token, price comparisons, evidence links and JSON export. A synthetic example is available without credentials. Live data comes from a server-side CoinGecko integration.

What is outside the current product

There is no trade execution, wallet signing, token custody, automated copy bot, predictive signal, complete portfolio return, fee-adjusted P&L or confirmed copy-relationship detector. The earlier pool-flow monitoring screens are a different prototype and are not the algorithm documented here. No claim of market uniqueness or demonstrated customer demand follows from this implementation.

2. Website workflow

From two addresses to a review the user can inspect

On initial load, the page shows a clearly marked synthetic example. The wallet fields are blank and the dates default to the last seven UTC calendar days. The page asks the server whether an API key is configured. Users can return to the example through [Try an example](#).

Step	Website behavior
1. Enter addresses	Wallet you followed becomes the leader input; Your wallet becomes the follower input. Both are public addresses, not authentication credentials.
2. Select dates	The requested dates use UTC. Today ends at the request time rather than at a future midnight.
3. Compare wallets	Client validation runs, controls lock during the request, and JSON is sent to <code>/api/compare</code> .
4. Read the summary	Leader and follower mean returns and their percentage-point difference use only comparable token rows.
5. Open a token	The detail panel explains the finding and shows average prices. The Evidence tab exposes supporting fills.
6. Export	A JSON download captures the report currently displayed, including mode, coverage, inputs and normalized trades.

Reading the result correctly

A negative gap means the follower return is lower. It does not mean the follower necessarily lost money: a follower at +8% and a leader at +10% have a -2 percentage-point gap. A row can be visible yet excluded from the headline; its finding explains why. An empty comparable set produces unavailable values rather than a fabricated zero.

Failure behavior

If a new comparison fails, the previous review remains visible with an error explaining that it is the previous result. This prevents a failed fetch from silently replacing a valid review. The browser has a 35-second request timeout; the server data request budget is 25 seconds. A ref-based lock prevents overlapping comparisons in the same page instance.

The status route checks only whether the key exists. Consequently the current Live data connected label is not evidence of a successful provider request, plan entitlement or fresh data. A successful comparison is stronger evidence. A failed status fetch can leave the initial checking state visible.

3. Architecture and source map

React interface, server boundary, provider adapter and pure analysis

```
Browser: app/page.tsx
  | GET /api/status          -> key-presence flag only
  | POST /api/compare        -> addresses + UTC dates
  v
Server: app/api/compare/route.ts
  validate -> read server secret -> shared 25-second budget
  |
  v
fetchHistory(leader)    fetchHistory(follower)
  | sequential pages    | sequential pages
+----- CoinGecko Pro wallet-trades endpoint -----+
  | normalized History  | normalized History
+-----+
  v
compare() -> positions() -> eligibility -> price attribution
  |
  v
Report JSON -> summary / token details / evidence / export
```

Architecture derived from app/page.tsx, API route and lib modules.

File	Responsibility
app/page.tsx	Client state, forms, request lifecycle, report presentation, export and optional browser tools.
lib/coingecko.ts	Address/date validation, provider request construction, pagination and trade normalization.
lib/analysis.ts	Types, position reconstruction, eligibility, returns, attribution and display formatting.
lib/demo.ts	Synthetic records for the example review.
app/api/compare/route.ts	HTTP checks, server secret access, parallel retrieval and error mapping.
app/api/status/route.ts	No-store response containing configured: boolean.
vite.config.ts / build/sites-worker.ts	Vite/vinext and Cloudflare/Sites runtime integration.
tests/analysis.test.ts	Deterministic logic, validation, normalization and mocked provider tests.

The package snapshot declares React 19.2.6, Next 16.3.4 APIs, vinext 1.0.0-beta.5, Vite 8.0.13, TypeScript 5.9.3 and Tailwind 4.2.1. The runtime uses a Cloudflare Worker environment through the Sites build. Dependency presence alone does not imply a feature: scaffold database and connector files are not a comparison ledger. The inspected comparison path does not write reports to a database.

4. Input validation and time boundaries

Reject malformed requests before requesting market data

```
const alphabet =
  "123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz";
let n = 0n;
for (const c of v) n = n * 58n + BigInt(alphabet.indexOf(c));
let bytes = 0;
while (n > 0) { bytes++; n >>= 8n; }
return bytes + (v.match(/^1*/)?.[0].length || 0) === 32;
```

lib/coingecko.ts, validAddress; excerpt reformatted after its type/regex guard.

The initial guard accepts only a 32-44 character base58 string. This excerpt then decodes its numeric value using `BigInt`. Leading 1 characters represent zero bytes and are added back to the byte count. Exactly 32 decoded bytes are required. This is structural validation, not proof that an address exists, belongs to the user, or is controlled by a particular trader. Identical leader and follower strings are rejected.

```
const start = Date.parse(from);
const end = Date.parse(to) + 86400000 - 1000;
if (end < start || end - start >= 30 * 86400000)
  throw new Error("Choose an ordered date range of no more than 30 days.");
// Other guards reject invalid calendar dates and future dates.
return { leader: b.leader, follower: b.follower,
  from, to, start, end: Math.min(end, now) };
```

lib/coingecko.ts, validateRequest; shortened excerpt, with intervening guards noted.

Dates must match YYYY-MM-DD and survive an ISO round trip, which rejects impossible calendar dates. Date-only parsing establishes UTC midnight. Adding a day and subtracting one second makes a past end date inclusive through 23:59:59. A 30-day inclusive span is accepted; a 31-day span is not. Future dates are rejected and today is capped at now.

Units and reproducibility

JavaScript uses milliseconds internally. The provider query uses whole UNIX seconds, obtained by flooring `start/1000` and `end/1000`. Normalized trades return to milliseconds through `Date.parse`. A regression test protects this conversion because a full ISO string with milliseconds previously caused a provider rejection.

The exported report preserves date strings and `createdAt`, but not the exact capped fetch end, provider cursor chain or raw responses. A same-day query made later can return additional fills. Reproducing a saved report means recalculating from its saved normalized trades, not expecting a fresh API request to be identical.

5. CoinGecko integration and pagination

A bounded fetch of observed swaps, not a complete wallet ledger

```
url.searchParams.set("from", String(Math.floor(from / 1000)));
url.searchParams.set("to", String(Math.floor(to / 1000)));
url.searchParams.set("per_page", "300");
if (cursor) url.searchParams.set("cursor", cursor);
const response = await fetcher(url, {
  headers: { "x-cg-pro-api-key": key, accept: "application/json" },
  signal,
  cache: "no-store"
});
```

lib/coingecko.ts, fetchHistory; exact operations with line breaks expanded.

The fixed upstream path is `/api/v3/onchain/networks/solana/wallets/{address}/trades` on `pro-api.coingecko.com`. The browser never constructs an authenticated provider request. The server adds the key in a header, avoiding a key in a browser URL. The provider documents wallet-trade date windows and opaque cursor pagination [1]; its authentication guidance supports backend header authentication [2].

Control	Implemented behavior
Page size / cap	300 records per page, at most five pages per wallet.
Concurrency	Two wallet fetches start together; pages within each wallet are sequential.
Deduplication	A Set of raw record IDs removes overlaps before normalization.
End condition	<code>meta.next_cursor === null</code> marks the provider page sequence complete.
Truncation	A repeated immediate cursor or the five-page cap returns <code>complete: false</code> .
Response validation	Require an array of at most 300 data items and a null or string <code>next_cursor</code> .
Rejected records	Invalid or out-of-range normalized records increment skipped.
Budget	At most 1,500 raw records per wallet and ten upstream calls per comparison.

The page cap is an application safety bound, not a provider guarantee that 1,500 records cover the period. complete means the returned page sequence ended under these rules. It cannot establish that every venue, transfer, token event or historical holding is represented. A longer cursor cycle eventually reaches the page cap.

Errors and availability

Provider 401/403 errors produce an access/plan message; 429 produces a rate-limit message. Other HTTP failures produce a generic provider error. The route maps these failures to HTTP 502. There are no automatic retries, exponential backoff or application result cache. Both fetches share a 25-second timeout; a rejected promise does not explicitly cancel its sibling immediately.

6. Normalization and data contracts

Convert provider records into the smallest useful analysis model

```
const buy = a.kind === "buy";
const token = a[buy ? "to_token_address" : "from_token_address"];
const quantity = Number(a[buy ? "to_token_amount" : "from_token_amount"]);
const usd = Number(a.volume_in_usd);
const time = Date.parse(String(a.block_timestamp));
// Guards require valid token/transaction strings and positive finite values.
return { id: raw.id, token, side: a.kind, quantity, usd, time,
        tx: a.tx_hash };
```

lib/coingecko.ts, normalizeTrade; selected statements, reformatted.

A buy tracks the acquired token and its quantity; a sell tracks the disposed token and its quantity. USD volume supplies the value used by the analysis. The adapter trusts the provider kind and token-leg semantics; it does not decode Solana instructions independently. Invalid records return null. Zero or missing USD valuations are not treated as free trades.

Type / fields	Meaning
Trade: id, token, side	Provider record identity, exact token address, buy or sell.
Trade: quantity, usd, time, tx	Chosen token units, USD volume, millisecond timestamp, transaction identifier.
History: trades, complete, skipped, pages	Normalized records and the retrieval-quality metadata used by the eligibility gate.
Position	Per-token fills, bought/sold totals, mean prices, timestamps, status and nullable return.
Comparison	Two optional positions, eligibility, finding, delays, effects and nullable gap.
Report	Mode, input addresses/dates, rows, mean returns, gap, eligible count, coverage and createdAt.

Precision and identity

Quantities and USD values are JavaScript numbers, using IEEE-754 binary floating point. The program does not retain integer base units or arbitrary-precision decimal arithmetic. A relative quantity tolerance is used when deciding whether inventory has closed. Display rounding occurs after analysis and should not be used to reproduce exact calculations.

Token addresses remain case-sensitive. Trade IDs, not transaction hashes alone, are deduplication keys because a transaction may contain multiple swap records. Live labels are abbreviated token addresses; the adapter does not fetch token names. Pool and DEX metadata, provider price fields and raw response bodies are not retained in Trade. The export is therefore an analysis snapshot rather than a raw provider archive.

7. Reconstructing observed positions

One token, one observed round trip, conservative exclusions

`positions()` removes duplicate IDs and invalid numeric records, groups by exact token address, and sorts each group by time then ID. It accumulates inventory from observed buys and sells. Starting inventory is assumed to be zero for this reconstruction; negative inventory flags an unknown cost basis rather than inventing missing purchases.

```
const tolerance = Math.max(bought, sold) * 1e-7;
let inventory = 0, cycles = 0, unknown = false, active = false;
// Inside the sorted-fill loop:
if (t.side === "buy") {
  if (!active) { cycles++; active = true; }
  inventory += t.quantity;
} else {
  inventory -= t.quantity;
  if (inventory < -tolerance) unknown = true;
  if (Math.abs(inventory) <= tolerance) {
    inventory = 0; active = false;
  }
}
```

`lib/analysis.ts, positions`; excerpt omits the timestamp-side check described below.

The tolerance is 0.00001% of the larger total quantity. Returning within that tolerance of zero closes the active inventory cycle. A new buy after closure starts another cycle. The code separately tracks sides at identical timestamps; opposing fills sharing a timestamp mark ordering as uncertain even though ID sorting provides a deterministic display order.

Status, in priority order	Condition and consequence
unknown	Negative observed inventory, no buys, or opposing fills at the same timestamp. No return.
multiple	More than one observed inventory cycle. No combined return.
closed	Bought and sold totals agree within tolerance. A gross return may be computed.
partial	Some sells, but observed bought quantity remains. No full-position return.
open	Observed buys without a completed exit. No full-position return.

The algorithm permits multiple buys and sells within a single active cycle. It does not implement tax lots, FIFO/LIFO or matching several separate round trips. remaining is $\max(0, \text{bought} - \text{sold})$ in the returned records, not the actual on-chain balance. Transfers, starting holdings and untracked venues can invalidate a ledger interpretation even when the observed totals balance.

8. Matching and eligibility

A candidate relationship with explicit gates

```
const complete = leaderHistory.complete && followerHistory.complete
  && !leaderHistory.skipped && !followerHistory.skipped;
const entryDelay = l?.firstBuy != null && f?.firstBuy != null
  ? (f.firstBuy - l.firstBuy) / 1000 : null;
const plausible = entryDelay !== null
  && entryDelay >= 0 && entryDelay <= 1800;
const comparable = complete && plausible
  && l?.returnPct != null && f?.returnPct != null;
```

lib/analysis.ts, compare; exact conditions with expanded formatting.

The row set is the union of both wallets' token addresses. Eligibility requires the same exact token, both reliable closed-position returns, and a follower first buy between zero and 1,800 seconds after the leader first buy. This 30-minute horizon is a fixed heuristic in this version. There is no statistical confidence model, but relationship lookup or proof of copy intent.

The global coverage gate

If either history is truncated or either wallet has even one skipped record, all token comparisons are suppressed. This is intentionally conservative: the application cannot assign an unusable record safely to a particular position. It also means one malformed record can hide an otherwise useful pair. Narrowing the dates can help, but does not repair provider coverage.

Finding	Interpretation
Incomplete history	The global coverage gate failed.
No follower record / Follower only	The token was observed in just one supplied history.
Unknown cost basis	Inventory or timestamp ordering cannot support a return.
Multiple round trips	The current per-token model cannot isolate the intended position.
Uncertain match	First buys fall outside the allowed ordering or horizon.
Partly exited / Position still open	At least one observed position is not eligible as closed.
Similar / Higher or better entry / Lower or better exit	Eligible pair; label comes from the arithmetic decomposition.

These checks are evaluated in the order shown. A coverage failure can therefore take precedence over a missing-wallet finding. A match can be coincidental; a genuine copy outside the horizon can be excluded. The rule is an auditable filter, not a discovered causal relationship. Exit delay is reported but is not an eligibility threshold.

9. Return and attribution mathematics

Exactly what the headline and explanation measure

For a token, let B be total bought quantity, S total sold quantity, U_{buy} the sum of buy USD volumes and U_{sell} the sum of sell USD volumes. Average entry $E = U_{buy} / B$ and average exit $X = U_{sell} / S$. These are quantity-weighted average fill prices, not averages of displayed prices. For an eligible closed position, $R = 100 * (X / E - 1)$.

```
entryEffect = (l.exit / f.entry - l.exit / l.entry) * 100;  
exitEffect  = (f.exit / f.entry - l.exit / f.entry) * 100;  
gap = f.returnPct - l.returnPct;
```

`lib/analysis.ts`, compare; source formulas, with non-null assertion omitted for readability.

Use EL and XL for the leader prices, EF and XF for the follower prices. Entry effect changes the entry from EL to EF while holding XL fixed. Exit effect then changes the exit from XL to XF while holding EF fixed. Adding the effects cancels XL/EF , leaving $100 * (XF/EF - XL/EL)$, exactly the follower return minus the leader return apart from floating-point rounding.

Why this is an attribution convention

This is an entry-first counterfactual decomposition. A different order of substitution can allocate the components differently while preserving the same total gap. It does not show that either wallet could have obtained the other's fill, nor identify whether market movement, slippage, routing, fees or a bot setting caused a price difference.

The larger absolute component selects the explanation; ties select exit. A gap with absolute value below 0.05 percentage points is labeled Similar execution. The phrase essentially the same average entry price uses a separate 0.05% relative entry-price threshold. These are display heuristics, not significance tests.

Aggregation and units

For N comparable token rows, the headline leader return is $\text{sum}(RL)/N$, and the follower return is $\text{sum}(RF)/N$. Every eligible token has equal weight regardless of trade size. Headline gap is the difference of these means. A percentage point is an arithmetic difference between percentages: -3.2% minus -2.4% is -0.8 pp. This is not a portfolio return, a dollar loss or a fee-adjusted result.

Gas, priority fees, tips and platform fees are not explicitly ingested or deducted. The app uses the provider's USD trade values as supplied. Changes in trade size alone do not change the price-based return when mean prices remain the same, although real dollar exposure changes.

10. Saved live example

One eligible token; observable exit-price difference

The saved report compares two public addresses on 6 October 2026. They were used as test inputs; neither ownership by the user nor a real copying relationship has been verified. Wallet A occupies the leader field and wallet B the follower field. Both page sequences ended after one page with zero skipped records. The report contains one comparable token.

Metric	Wallet A	Wallet B
Observed buys / sells	6 buys / 1 sell	2 buys / 1 sell
Bought and sold quantity	617,404.703945	1,833,120.600846
Average entry, USD	0.00009246456454	0.00009245858084
Average exit, USD	0.00009021965180	0.00008947622908
Gross return	-2.4278627681%	-3.2256084182%

Derived quantity	Value
First-buy delay	505 seconds (8 minutes 25 seconds)
Last-sell delay	705 seconds (11 minutes 45 seconds)
Entry-price effect	+0.0063146353 percentage points
Exit-price effect	-0.8040602854 percentage points
Total gap	-0.7977456501 percentage points

The entry prices are nearly identical, with a small favorable entry component for B. The larger unfavorable exit component produces the Lower exit price finding. The UI rounds this to leader -2.4%, follower -3.2% and gap -0.8 pp. The 505-second entry difference does not prove a delay caused the outcome.

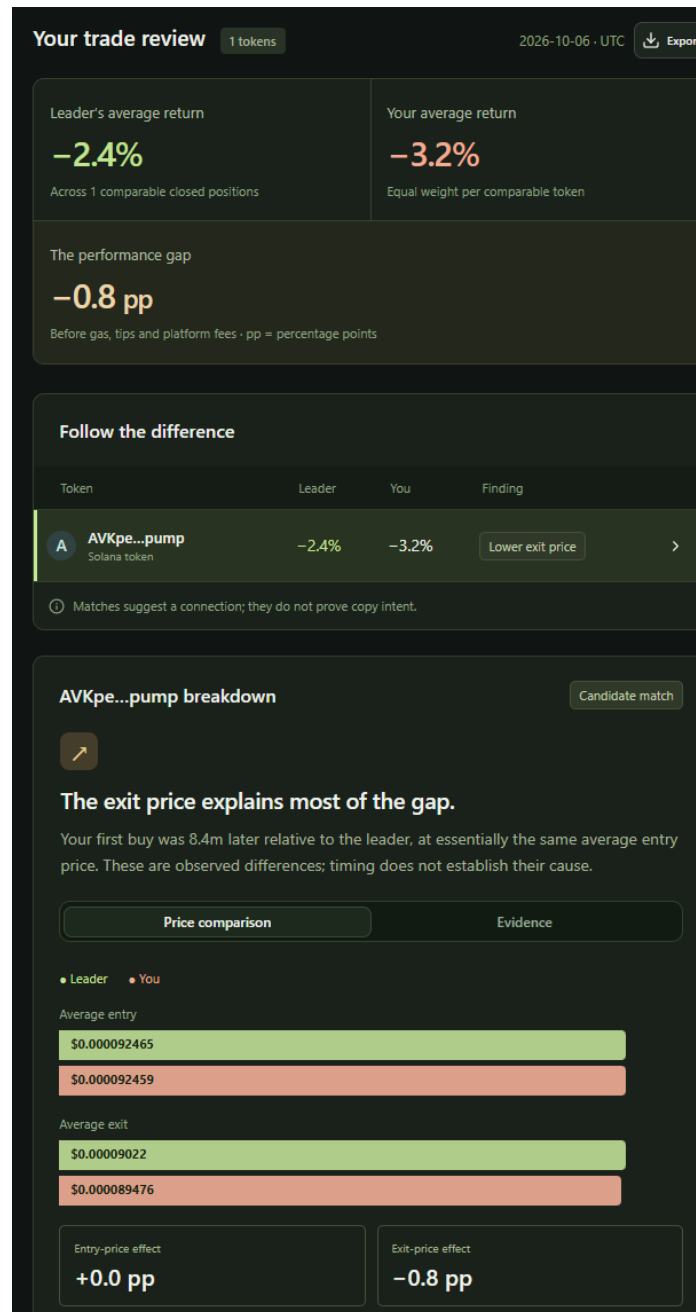
Evidence and reproducibility

The package includes evidence/verified-live-report.json and the three supplied screenshots. The report was created at 2026-10-06T09:59:13.891Z. This is a saved observation, not a promise that a later request yields identical data. The full public addresses, token identifier and transaction hashes are retained in the JSON for inspection. Recompute the formulas from the unrounded values; do not use screenshot rounding.

This case demonstrates an end-to-end returned-data comparison. It does not validate every supported venue, independently verify all USD valuations, or establish net dollar P&L. It should be described in an article as a test-wallet example rather than a confirmed copy-trading failure.

11. The example as presented on the website

Saved interface evidence, not a new live query



Saved screenshot: summary, token finding and average-price comparison. The full-resolution image is included in the evidence folder.

The evidence panel is the route from an explanation to individual records. Price bars compare averages; they are not a historical market-price chart. Their UI width has a minimum of 25%, so printed values should be used for quantitative interpretation. The interface does not display an account balance or make a prediction.

12. HTTP contracts and error behavior

The browser-facing API is small and deterministic

GET /api/status

```
{"configured": true}
```

Example response; boolean reflects server key presence only. Cache-Control: no-store.

POST /api/compare

```
{
  "leader": "<32-byte base58 public address>",
  "follower": "<different 32-byte base58 public address>",
  "from": "2026-10-06",
  "to": "2026-10-06"
}
```

Request shape; placeholders must be replaced. Content-Type: application/json and matching Origin required.

A successful response is a Report, not a provider pass-through: mode, from, to, leaderWallet, followerWallet, rows, leaderAverage, followerAverage, gap, comparableCount, coverage and createdAt. Returns and effects are numbers or null. Null means unavailable; consumers must not coerce it to zero. The companion saved JSON is a complete successful response example.

HTTP status	Condition
200	Analysis returned, even if no rows are eligible; inspect comparableCount and coverage.
400	Malformed JSON or address/date validation error.
403	Origin missing or not equal to the request URL origin.
413	Declared Content-Length or read text length exceeds 4,096.
415	Content-Type does not start with application/json.
503	COINGECKO_API_KEY is absent from the server environment.
502	Provider access/rate-limit errors, unexpected data, timeout or another retrieval failure.

Errors use {"error": "message"}; successful and error responses use Cache-Control: no-store. The request-text check is based on JavaScript string length, while Content-Length describes bytes. The body is read before that second size check, so it is not a streaming byte limit. The fixed provider host avoids a user-supplied upstream URL.

This API has no explicit schema version field. A future contract change should introduce versioning and compatibility rules. Third-party consumers must satisfy the current Origin check, but that check is not authentication and must not be described as access control for arbitrary clients.

13. Interface state, export and browser tools

How the client presents and exposes the current report

```
const [leader, follower] = await Promise.all([
  fetchHistory(parsed.leader, parsed.start, parsed.end, key, signal),
  fetchHistory(parsed.follower, parsed.start, parsed.end, key, signal)
]);
return json(compare(leader, follower, {
  mode: "live", from: parsed.from, to: parsed.to,
  leaderWallet: parsed.leader, followerWallet: parsed.follower
})));
```

app/api/compare/route.ts; retrieval and report construction, reformatted.

The browser owns the form, busy flag, error message, selected token and current Report. compareWallets validates the form, locks requests, submits JSON, then updates the report and selected token together. Its catch path keeps the old report. The server computes every live finding, so browser presentation does not require access to the provider secret.

Export and transaction evidence

Export serializes the current Report with `JSON.stringify(report, null, 2)`, creates a JSON Blob, downloads through an object URL and later revokes that URL. The filename includes mode and from date. Export does not request fresh data and is not an upload or server-side save. Keep mode when using exported examples so synthetic and live data remain distinguishable.

Live transaction identifiers become encoded Solscan transaction URLs opened in a new tab with `noreferrer`. Demo records do not receive real transaction links. Amounts and timestamps in Evidence are observed values, not an independent explorer reconciliation performed by CopyCheck.

Optional WebMCP integration

Where `document.modelContext` is available, the page registers three browser tools. `load_demo_review` changes the current view to the example; `compare_wallets` validates its four arguments and runs the same comparison path; `get_current_review` reads a summary of the visible review. Registration is conditional and cleaned up on unmount. A normal browser without this API still uses the ordinary interface.

Tool	Effect
load_demo_review	Loads synthetic data; no provider request.
compare_wallets	Accepts leader, follower, from and to; updates UI and can consume provider requests.
get_current_review	Returns mode, dates, eligible count, gap and token findings; not the full export.

This integration is not an embedded LLM analysis service. The explanatory text is chosen from deterministic templates in `compare()`, and calculations remain local to the application server code.

14. Security, privacy and operational limits

Implemented safeguards and the gaps that remain

Area	Current implementation / practical limit
Credentials	Server-only COINGECKO_API_KEY binding; header authentication to the provider. Never enter it into either wallet field.
Wallet access	Public addresses only. No signing, private keys, seed phrases or funds are requested.
Request checks	Origin, content type, body length and address/date validation. These do not replace authentication.
Storage	No application database write in the comparison path. Browser state and downloaded reports still contain addresses.
Provider disclosure	Addresses and date bounds are sent to CoinGecko. Hosting/provider logs may retain operational information.
Abuse controls	No application-level authentication, user quota or rate limiter in these routes.
Resource bounds	Page and timeout caps bound one request; they do not bound aggregate public traffic.
Observability	No dedicated application usage meter, audit log or provider-health dashboard in the inspected feature.

Production hardening priorities

The published interface is public. An attacker can call its server endpoint repeatedly and consume the owner's provider allowance. The same-origin comparison reduces some unwanted browser calls but arbitrary clients can supply an Origin header. Add server-enforced rate limits, usage budgets and monitoring before treating the service as ready for unrestricted traffic. Key rotation and provider account restrictions belong in the operational process.

A stronger version would enforce byte limits while streaming, validate output shape at the client boundary, distinguish provider rate limiting in HTTP status, and record exact fetch cutoffs. These are recommendations, not features already implemented. Do not claim that the page's busy state prevents concurrent requests from other tabs or clients.

Scientific and financial limits

Data completeness is conditional on provider coverage. USD valuations are provider-supplied, numbers use floating-point arithmetic, and fees/transfers/prior holdings are outside the model. A price decomposition is not causal evidence. There is no claim of audited accuracy, profitability, execution improvement, or suitability for tax reporting. These boundaries determine what evidence is needed before expanding the product's claims.

15. Local setup and operating runbook

Developer instructions for the inspected repository

Use the existing project checkout and lockfile. package.json requires Node.js 22.13.0 or newer. Install its dependencies in a supported development environment; retain the Sites/vinext configuration rather than replacing the build with a generic Next.js deployment.

```
npm ci
# Create an ignored .dev.vars file for local Worker development:
# COINGECKO_API_KEY=YOUR_SERVER_SIDE_KEY
npm run dev

# Deterministic checks; do not require a provider key:
node --experimental-strip-types --test tests/analysis.test.ts
npx tsc --noEmit

# Build using the repository's framework wrapper:
npm run build
```

Run from the project root. Placeholder only; no actual credential is shipped.

For local development, place the value directly after the equals sign in .dev.vars and do not commit that file. For the hosted site, configure the same variable as a server secret through the hosting environment and publish the corresponding deployment. A local file edit alone does not update the deployed Worker. Wallet data requires a key entitled to this endpoint; key presence is insufficient.

Check	Expected result / response
Open website	Synthetic example is labeled and readable before a live request.
GET /api/status	configured is true only when the runtime binding exists.
Compare test addresses	A live banner and coverage metadata appear after a successful response.
No comparable rows	Inspect row findings, range and coverage. This is not automatically an API failure.
Access error	Check server binding and wallet-trades entitlement, not the browser wallet fields.
Rate-limit error	Wait, reduce traffic or add server-side controls; no automatic retry is implemented.
Incomplete history	Try a shorter range. A high-activity wallet can exceed five pages.
Old result after error	Confirm the error and previous-review notice before interpreting the visible numbers.

The starter includes environment-specific wrappers and deployment scaffolding. Local platform prerequisites can differ; build success must be verified in the actual hosting environment. Avoid using a stale README claim as proof of current live status. The source version and saved evidence in this package are the basis for this document.

16. Tests, reproducibility and acceptance

What has been checked and what still needs testing

During this documentation run, node `--experimental-strip-types --test tests/analysis.test.ts` completed with 14 passes and zero failures. These tests exercise core logic with synthetic data and mocked fetch responses. They do not require a live API key. The saved live report provides separate evidence of a successful provider-backed user workflow.

Test area	Assertions covered
Mathematics	Demo returns reconcile with attribution; price return is invariant to proportional size changes.
Position eligibility	Partial exits, prior holdings, multiple cycles and opposing same-second fills are withheld.
Matching and identity	Incomplete histories suppress comparisons; token case and duplicate IDs are handled; time horizon excludes mismatches.
Validation	Base58 addresses decode to 32 bytes; calendar/date constraints and identical wallets are checked.
Normalization	Chosen token quantities and USD volumes are used; missing valuations are rejected.
Pagination	Overlap deduplication, cursor forwarding and five-page truncation behavior are checked.
Regression and errors	Date parameters use whole UNIX seconds; upstream errors do not expose credentials.

Recompute the saved case

Open `evidence/verified-live-report.json`. For each position, sum buy USD values and divide by total buy quantity to recover entry; repeat for sells to recover exit. Apply the equations in section 9. The entry and exit effects should sum to `-0.7977456501 pp` within numerical tolerance. Both histories are marked complete, skipped is zero, and `entryDelay` is 505, so the candidate satisfies the current gates.

Acceptance criteria for changes

Before changing the engine, require an explanation for every withheld result, preservation of null as unavailable, consistent UTC handling, and arithmetic reconciliation before display rounding. Test both a successful request and a provider failure in the UI; confirm the previous review is not mislabeled as newly fetched. Verify that secrets never appear in responses or client assets.

Current tests are not a security audit or exhaustive integration suite. Gaps include route-level HTTP tests, browser automation across every error state, property-based numeric testing, load/abuse tests and independent transaction/valuation reconciliation. Multiple-position matching and fee accounting would require new data models and tests rather than just an extra UI label.

17. Roadmap and glossary

Proposals are not part of the current implementation

Proposed work	What it would require
Dollar P&L	Explicit costs and proceeds model, treatment of fees, transfers, starting inventory and partial lots; clear gross/net labels.
Multiple round trips	Position segmentation and pair matching instead of one row aggregating every token fill.
Better provenance	Schema/algorithm version, exact request cutoff, source IDs and reproducible snapshot metadata.
Operational protection	Rate limiting, quotas, provider-health checks and server-side usage monitoring.
More accurate arithmetic	Integer base-unit quantities or decimal arithmetic, with migration and regression tests.
Cause investigation	Submission/confirmation times, quotes, routes and bot logs. Timestamp correlation alone is insufficient.
Additional networks	Network-specific validation, provider capability checks and correctly scoped address identity.

Glossary

Term	Meaning in CopyCheck
Leader / follower	User-selected roles for the two addresses, not verified identity or relationship.
Observed position	Inventory reconstructed from returned swaps for one token in the chosen range.
Closed	Observed bought and sold quantities agree within tolerance and other status checks pass.
Comparable	Both closed returns pass coverage and first-buy matching gates.
Gross return	Average exit / average entry minus one, expressed as a percentage; fees not explicitly deducted.
Percentage point (pp)	Unit for subtracting percentage returns and reporting the price effects.
Complete history	Provider pagination ended within app limits; not proof of a full wallet ledger.
Candidate match	Exact token and accepted first-buy timing, without proof of copying.

The product's current value is a reproducible explanation of observed execution differences. Further features should preserve the distinction between data, arithmetic, inference and claims about causes.

18. References and document provenance

Sources, code excerpts and packaged evidence

[1] CoinGecko, Wallet Trades endpoint documentation. <https://docs.coingecko.com/reference/wallet-trades> - consulted 6 October 2026. Used for the integration contract; the implementation imposes its own page cap and validation rules.

[2] CoinGecko, Authentication documentation. <https://docs.coingecko.com/reference/authentication> - consulted 6 October 2026. Used for Pro host and server-side header authentication guidance.

[3] CopyCheck source snapshot `ca57fc0758fea5e20fc5f31512b45592fad02a66`. Primary implementation evidence: `app/page.tsx`; `app/api/compare/route.ts`; `app/api/status/route.ts`; `lib/coingecko.ts`; `lib/analysis.ts`; `lib/demo.ts`; `tests/analysis.test.ts`; `package.json`; `vite.config.ts`; `build/sites-worker.ts`.

[4] Saved live report, created `2026-10-06T09:59:13.891Z`, and three user-supplied interface screenshots. See `evidence/` in this documentation package. Their provenance and article notes are retained alongside the files.

Code excerpt policy

Excerpts in this paper preserve the source logic but expand compressed statements for readability. Omissions are labeled; they are not standalone replacement implementations. `Developer-reference.html` contains complete line-numbered copies of the selected application modules from the inspected checkout, allowing readers to verify context. Dependency source, credentials and ignored environment files are excluded.

Package contents

Artifact	Purpose
<code>CopyCheck-white-paper.pdf</code>	Shareable explanation of product behavior, architecture, algorithms and constraints.
<code>Developer-reference.html</code>	Self-contained browsable paper plus annotated module map and line-numbered principal source.
<code>evidence/</code>	Saved JSON, screenshots and provenance notes for the example.
<code>source/</code>	Selected source snapshot for the documented modules; not a full deployable checkout.
<code>MANIFEST.json</code>	Source commit, SHA-256 hashes of included source/evidence and validation summary.

Prepared from the inspected repository and saved evidence on 6 October 2026. This is implementation documentation rather than a guarantee of future provider capabilities or future deployed behavior. Revalidate it when the analysis rules, data contract or deployment changes.